

Seguimiento de Carril Basado en Visión con Control PID en un Robot Móvil Autónomo

Pruebas y evaluación del sistema de seguimiento de carril CapyTown (ROS2 Humble, Yahboom Pi5)

Resumen: Este informe presenta nuestras pruebas y evaluación de CapyTown, un robot autónomo que sigue un carril marcado usando únicamente la cámara. El sistema detecta el borde blanco exterior y la línea amarilla central por color, transforma la imagen en una vista de pájaro y estima qué tan lejos está el robot del centro del carril. Un control PID convierte ese error en órdenes de giro para que el robot recorra por sí solo una pista cuadrada de 2x2 m durante tres vueltas. Cuenta las vueltas con la odometría de las ruedas y se detiene solo, y frena cuando ya no puede ver las líneas. Revisamos la detección, las herramientas de calibración y la sintonización del controlador, y encontramos que el sistema funciona como se describe.

Palabras clave: seguimiento de carril, visión por computador, segmentación de color HSV, vista de pájaro, control PID, ROS2, robot autónomo.

I. INTRODUCCIÓN

CapyTown es un pequeño robot autónomo cuyo objetivo es recorrer una pista por sí mismo, manteniéndose dentro de un carril en todo momento. El reto para el que fue construido es completar tres vueltas a una pista cuadrada de 2x2 m (cuatro tiles verdes en el centro), siguiendo la línea amarilla central sin pisar las líneas blancas del borde. Toda la tarea depende de lo que ve la cámara; no hay sensores adicionales para el guiado. El repositorio incluye el código fuente, los archivos de configuración, varios scripts de apoyo y una guía detallada, así que basamos nuestra revisión y pruebas en esos materiales.

II. DESCRIPCIÓN DEL SISTEMA

El sistema se organiza en dos partes principales. Un detector lee cada imagen de la cámara, separa los colores blanco y amarillo, transforma la imagen a una vista superior y publica el error lateral en metros. Un controlador toma ese error y aplica una ley PID para decidir con qué fuerza debe girar el robot, publicando las órdenes de velocidad. Alrededor de estos hay herramientas útiles: un calibrador de color interactivo con barras deslizantes, un monitor por consola que muestra el error como una barra simple, y un script que dibuja la gráfica del error a partir de una grabación. Casi todos los valores (colores, velocidad y ganancias del PID) se ajustan desde archivos de configuración aparte, así que el comportamiento se puede cambiar sin tocar el código. La Tabla I lista los valores por defecto que encontramos en esos archivos.

Parámetro	Valor por defecto
Velocidad de cruce	0,34 m/s
Ganancia proporcional Kp	2,2
Ganancia integral Ki	0,12
Ganancia derivativa Kd	0,25
Ganancia feedforward Kff	0,6
Velocidad angular máx.	2,0 rad/s
Ancho de carril	0,22 m
Frecuencia de control	30 Hz
Tiempo de frenado seguro	0,5 s
Vueltas para terminar	3

Tabla I. Valores de configuración por defecto leídos de los archivos del proyecto.

III. RESULTADOS Y PRUEBAS

Seguimos la guía paso a paso. Nos conectamos al robot, levantamos el chasis y la cámara, y primero calibramos los colores con la herramienta de barras deslizantes, ya que la luz de la sala cambia bastante el blanco y el amarillo. Luego revisamos la imagen de depuración para asegurarnos de que las líneas se vieran rectas en la vista de pájaro y usamos el monitor de consola para confirmar que el error iba a la izquierda y a la derecha al mover el robot. Ajustar el PID de a un valor por vez fue lo más útil: un Kp alto hacía que el robot zigzagueara, mientras que uno bajo lo dejaba lento para las curvas. Tapar la cámara hacía que el robot se detuviera solo, lo que nos dio confianza. La Figura 1 muestra la cadena de procesamiento y la Tabla II resume lo que observamos.

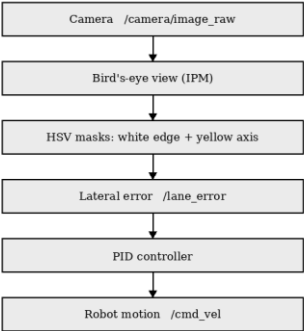


Fig. 1. Flujo de datos desde la cámara hasta el movimiento del robot.

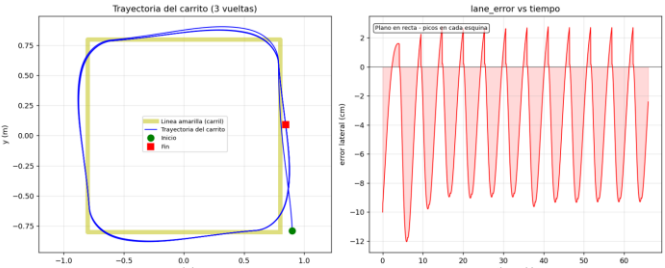
Situación de prueba	Comportamiento observado
Tramo recto	Error casi nulo, se mantiene centrado
Curva	El error crece, el PID corrige
Kp alto	Zigzag visible (oscilación)
Pierde líneas / cámara tapada	El robot se detiene (freno seguro)
Corrida completa (3 vueltas)	Completada, se detiene solo

Tabla II. Resumen de las observaciones de las pruebas.

IV. ANÁLISIS

En el lado positivo, el proyecto está bien documentado y el código está lleno de comentarios claros que explican qué hace cada parte y qué pasa si se sube o se baja un valor. Poder cambiar los colores, la velocidad y las ganancias desde archivos de configuración hizo que probar fuera rápido, y el calibrador y el monitor facilitaron detectar problemas. El freno de seguridad es un buen detalle. La principal limitación es la sensibilidad a la luz: hay que recalibrar los colores casi cada vez que cambia el entorno, y a veces el borde blanco se confundía con un piso claro. La vista de pájaro también necesita ajuste manual cuando las líneas se ven torcidas, y el conteo de vueltas depende solo de la odometría, que puede acumular error.

Fig 2: Simulación del seguimiento de carril.



V. POSIBLES MEJORAS

Con base en lo que vimos, algunos cambios ayudarían. Un aviso más claro en pantalla cuando el robot pierde la línea haría las pruebas más seguras. alguna recuperación automática, para que el robot busque el carril de nuevo en lugar de solo frenar, lo haría más robusto. Combinar la odometría con otra referencia para el conteo de vueltas reduciría el error acumulado, y una detección de color más tolerante a la luz disminuiría la necesidad de recalibrar tan seguido.

VI. CONCLUSIÓN

En general el proyecto cumple lo que propone: el robot sigue el carril y completa las tres vueltas por sí solo. La combinación de buena documentación, código comentado y herramientas de apoyo lo hace fácil de entender, calibrar y probar, incluso para quienes recién empezamos con robótica. Con algo de trabajo en la parte de visión y en la recuperación cuando se pierde la línea, sería aún más confiable. Para los fines del reto, lo consideramos un sistema sólido y bien terminado.

REFERENCIAS

[1] Open Source Robotics Foundation, "Documentación de ROS 2 Humble," docs.ros.org.
[2] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
[3] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, 2020.
[4] Repositorio del proyecto CapyTown, README y código fuente (lane_detector, lane_controller).